```
(*
                              CS51 Lab 12
                  Imperative Programming and References
 *)
(*
                                SOLUTION
 *)

(*======================================================================
Readings:

    This lab builds on material from Chapter 15 of the textbook
    <http://book.cs51.io>, which should be read before the lab session.

Objective:

    This lab provides practice with reference types and their use in
    building mutable data structures and in imperative programming more
    generally. It also gives further practice in using modules to
    implement abstract data types.

    There are three total parts to this lab. Please refer to the following
    files to complete all exercises:

    -> lab12_part1.ml -- Part 1: Fun with references (including `gensym`)
       lab12_part2.ml -- Part 2: Appending mutable lists
       lab12_part3.ml -- Part 3: Adding serialization to imperative queues
======================================================================*)

(*======================================================================
                               IMPORTANT

As usual, when implementing functions in this lab, you shouldn't feel
beholden to how the definition is introduced in the skeleton code
below. For instance, you might want to add a `rec`, or use a different
argument list, or no argument list at all but binding to an anonymous
function instead. THIS IS ESPECIALLY PERTINENT TO CERTAIN PROBLEMS IN
THIS LAB, WHERE THE NORMAL COMPACT NOTATION FOR INTRODUCING FUNCTIONS
MAY NOT BE APPROPRIATE.
 **********************************************************************)

(*======================================================================
Part 1: Fun with references (including `gensym`)

......................................................................
Exercise 1: Consider a function `incr` that takes an `int ref`
argument and has the side effect of incrementing the integer stored in
the `ref`. What is an appropriate type for the return value? What
should the type for the function as a whole be?
......................................................................*)

(* ANSWER: The function is called for its side effect, not to return
```

```
   an updated value, so the appropriate return type is `unit`. The type
   of the function is thus

     int ref -> unit
 *)

(*..................................................................
Exercise 2: Now implement the `incr` function.
..................................................................*)

let incr (n : int ref) : unit =
  n := !n + 1 ;;

(* As it turns out, the `incr` function is provided as part of the
   `Stdlib` module. Now you know how it can be implemented. *)

(*..................................................................
Exercise 3: Write a function `remember` that returns the last string
that it was called with. The first time it is called, it should return
the empty string.

    # remember "don't forget" ;;
    - : string = ""
    # remember "what was that again?" ;;
    - : string = "don't forget"
    # remember "etaoin shrdlu" ;;
    - : string = "what was that again?"

(This is probably the least "functional" function ever written!)

As usual, you shouldn't feel beholden to how the definition is
introduced in the skeleton code below.
..................................................................*)

let remember : string -> string =
  let memory = ref "" in
  fun (msg : string) ->
    let previous = !memory in
    memory := msg;
    previous ;;

(* An alternative is to move the `memory` variable outside the
   function definition, as:

    let memory = ref "" ;;

    let remember msg =
      let previous = !memory in
      memory := msg;
      previous ;;

   But this allows access to the contents of `memory` outside the
   `remember` function. But this is a failure of the abstraction that
   `remember` is intended to implement, violating the edict of
```

```
   prevention. In particular, it would allow someone to modify the
   `memory` value and so have the `remember` function return a value
   that was *not* the previous string argument.

   A common error is to start the function as would be typical given
   its signature:

    let remember msg =
    ...

   and then introduce the `memory` variable

    let remember msg =
      let memory = ref "" in
      ...

   finally finishing with the update and return value

    let remember msg =
      let memory = ref "" in
      let previous = !memory in
      memory := msg;
      previous ;;

   Unfortunately, this version introduces a new `memory` each time
   it's called, so always returns the empty string. *)

(* The `gensym` function (short for "GENerate SYMbol") has a long
history dating back to the early days of the programming language
LISP.  You can find it as early as the 1974 MacLISP manual (page
53). <https://url.cs51.io/lisp-gensym>

    (What is LISP you ask? LISP is an untyped functional programming
    language invented by John McCarthy in 1958, which he based
    directly on Alonzo Church's lambda calculus. It is one of the most
    influential programming languages ever devised. You could do worse
    than spend some time learning the Scheme dialect of LISP, which,
    by the way, will be made much easier by having learned a typed
    functional language -- OCaml.)

The `gensym` function takes a string and generates a new string by
suffixing a unique number, which is initially 0 but is incremented
each time `gensym` is called.

For example,

    # gensym "x" ;;
    - : string = "x0"
    # gensym "x" ;;
    - : string = "x1"
    # gensym "something" ;;
    - : string = "something2"
    # gensym (gensym "x") ;;
    - : string = "x34"
```

```
161
162 There are many uses of `gensym`, one of which is to generate new
163 unique variable names in an interpreter for a programming language. In
164 fact, you'll need `gensym` for just this purpose in completing the
165 final project for CS51, so this is definitely not wasted effort.
166
167 ..................................................................
168 Exercise 4: Complete the implementation of `gensym`. As usual, you
169 shouldn't feel beholden to how the definition is introduced in the
170 skeleton code below. (We'll stop mentioning this from now on.)
171 ..................................................................*)
172
173 let gensym : string -> string =
174   let suffix = ref 0 in
175   fun str ->
176     let symbol = str ^ string_of_int !suffix in
177     suffix := !suffix + 1;
178     symbol ;;
179
180 (* Taking advantage of the `incr` function implemented above and also
181    available as part of the `Stdlib` module, we can have
182
183     let gensym : string -> string =
184       let suffix = ref 0 in
185       fun str ->
186         let symbol = str ^ string_of_int !suffix in
187         incr suffix;
188         symbol ;;
189
190    An alternative implementation performs the incrementation before
191    the string formation, so that the return value need not be stored
192    explicitly. But the initial value for the counter must start one
193    earlier.
194
195     let gensym : string -> string =
196       let suffix = ref ~-1 in
197       fun str ->
198         incr suffix;
199         str ^ string_of_int !suffix ;;
200    *)
```